Presenter 3 — Aarush: encryption, decoding, and evidence (10:00–15:00)

Your question: If the note is encrypted, what can the receiver and guardian still do—and what have we really proved? You own **Locked note**, **Decode**, **Guide**, **Case closed**, and **Evidence**. Do not repeat the detector-rule comparison; Harsh has covered it.

Start screen: Guided case — Locked note. **End screen:** Evidence page, with the limits and next experiment stated aloud.

Your five-minute route

10:00–10:35 — Set up the new question

Point to the four-stage flow: note → lock → timing bits → receiver. Say:

> "Harsh showed that the guardian can notice timing. Now we ask a different question: can it read the message? We lock the text using AES-GCM in the browser, then model those encrypted bytes as short and long waits. The intended receiver knows the key phrase; the guardian sees the pauses, not the phrase."

Plain distinction: encryption hides *contents*. It does not automatically make a timing pattern look ordinary.

10:35–11:55 — Run the locked-note experiment

Leave `HI` and the prefilled demo key phrase. Click `Lock this note`. Point to **2 bytes → 46 bytes → 368 bits → 6 min 8 sec**. Point to **timing bits recovered**, the successful **HI** unlock, and the guardian result. Click **Try a wrong key**; it should say **Could not unlock**. If time permits, restore the receiver phrase by typing `cadence-demo-key`, then set **Network delay variation** to **50 ms** to show that damaged encrypted bits can also block the unlock.

> "The encryption is real browser cryptography. Even a two-letter note becomes a 46-byte package because it needs a salt, one-time value, and integrity check. At roughly one timing bit per second, that is over six minutes of modelled transmission. With the right key and intact bits, the receiver gets `HI`. A wrong key fails. The guardian may still flag the timing without knowing the note. Encryption changes secrecy, not the visibility or cost of this channel."

Read the guardian's flagged-chunk count from the screen; it can change when fresh random encryption bytes are made. **No actual ping is sent by this experiment.** The prefilled phrase is for demonstration, not a strong production secret.

11:55–12:35 — Show the original decoder

Click `Decode` in the top navigation. Point to **Try your own message**, **Choose a covert run**, the threshold choices, the recovered-bit result, and the sent/decoded bit strip. You can leave its defaults; the upper ideal example and the prepared noisy run intentionally have different outcomes.

> "Decode is the detailed receiver view. It turns a received short or long gap back into zero or one. The top toy example is clean and should recover the note. Below, a prepared run includes network variation, so we compare sent and recovered bits and try fixed or adaptive dividing lines. A high bit percentage is useful, but an encrypted message needs *every* bit intact to unlock."

Important: the guided locked note and this prepared decoder run are related demonstrations, not the same live packet trace.

12:35–12:55 — Show that the site teaches its vocabulary

Click `Guide`. Type `jitter` into **Find a word** and point to the explanation. Clear search only if you have time.

> "The guide explains the units, charts, and terms inside the product. Jitter simply means packet delay changes from one request to the next."

12:55–13:20 — Close the guided story

Click `Guided case`, then chapter `05 Case closed`. Point to the three summary cards.

> "The full story is sender, detective, guardian, and encrypted receiver. A message can be readable, detectable, both, or neither; a defence also has a cost. The final question is what our evidence actually supports."

Click `Inspect evidence and limits`.

13:20–14:35 — Make the evidence boundary explicit

On `Evidence`, point to the three counts and the three levels of evidence. Show **610 simulated recordings**, **0 real-capture labels**, and **0 unverified imports** on a fresh session. Point to **Turn the case into a measured result**. If time permits, scroll to the export buttons; do not start a download unless asked.

> "The software can read runs, calculate gaps, compare detectors, model pacing, and encrypt and unlock in the browser. The shipped examples are simulations. We have not included real receiver-side lab captures, so we do not claim measured real-network detection performance. Our next experiment is to capture ordinary and encoded pings at the receiver, repeat under delay and loss, and report detection, false alarms, recovered bits, and added delay."

14:35–15:00 — Give the architecture and final claim

> "Under the hood, Python tools generate or capture runs and export a versioned JSON bundle. The React/TypeScript browser reads that bundle, computes the analysis, draws the charts, and uses Web Crypto for the locked note. Our contribution is a coherent, inspectable attacker–defender learning and evaluation workflow. Real lab captures are the next validation step."

Stop there. Leave **Evidence** visible for questions.

Your three memory anchors

1. **Real encryption; modelled packet timing.**
2. **Correct key plus intact bits are required to recover the note.**
3. **Working software and simulated results are distinct from real-network proof.**

If you get a question

- **"Did you invent AES?"** No. The browser uses standard AES-256-GCM. Our project connects encryption to the timing-channel investigation.

- **“Why does `HI` take six minutes?”** The encrypted package has 46 bytes, or 368 bits. At about one bit per second, that is roughly 368 seconds. It is an estimate from the model.
- **“Can the guardian decrypt it?”** It does not have the key. It can still examine the timing metadata and raise an alarm.
- **“What if one bit flips?”** AES-GCM's integrity check rejects the whole package; bit recovery and whole-message recovery are different measures.
- **“Where is the backend?”** The live UI is a browser app reading a prepared data bundle. Python scripts handle generating/exporting examples and the planned lab capture workflow; the browser does the interactive analysis and encryption.

If the clock is running out: keep Locked note and Evidence. Show Decode and Guide for 10–15 seconds each, then deliver the final honest claim. Do not consume extra time trying to make random encrypted bytes produce a particular flagged count.